Unit-07
"Advance Foundation and Java Frameworks"

Topic List :-

- ❑ Process and Thread .
- ❑ Multithreading , Synchronization , Inter-Thread Communication .
- ❑ File Handling ,File class , Byte Streams, Character Streams.
- ❑ Buffered Streams , Serialization , Deserialization .
- ❑ Lambda Expression , Stream API, Optional Class.

Topic :-01
"Process and Thread "

Process :-

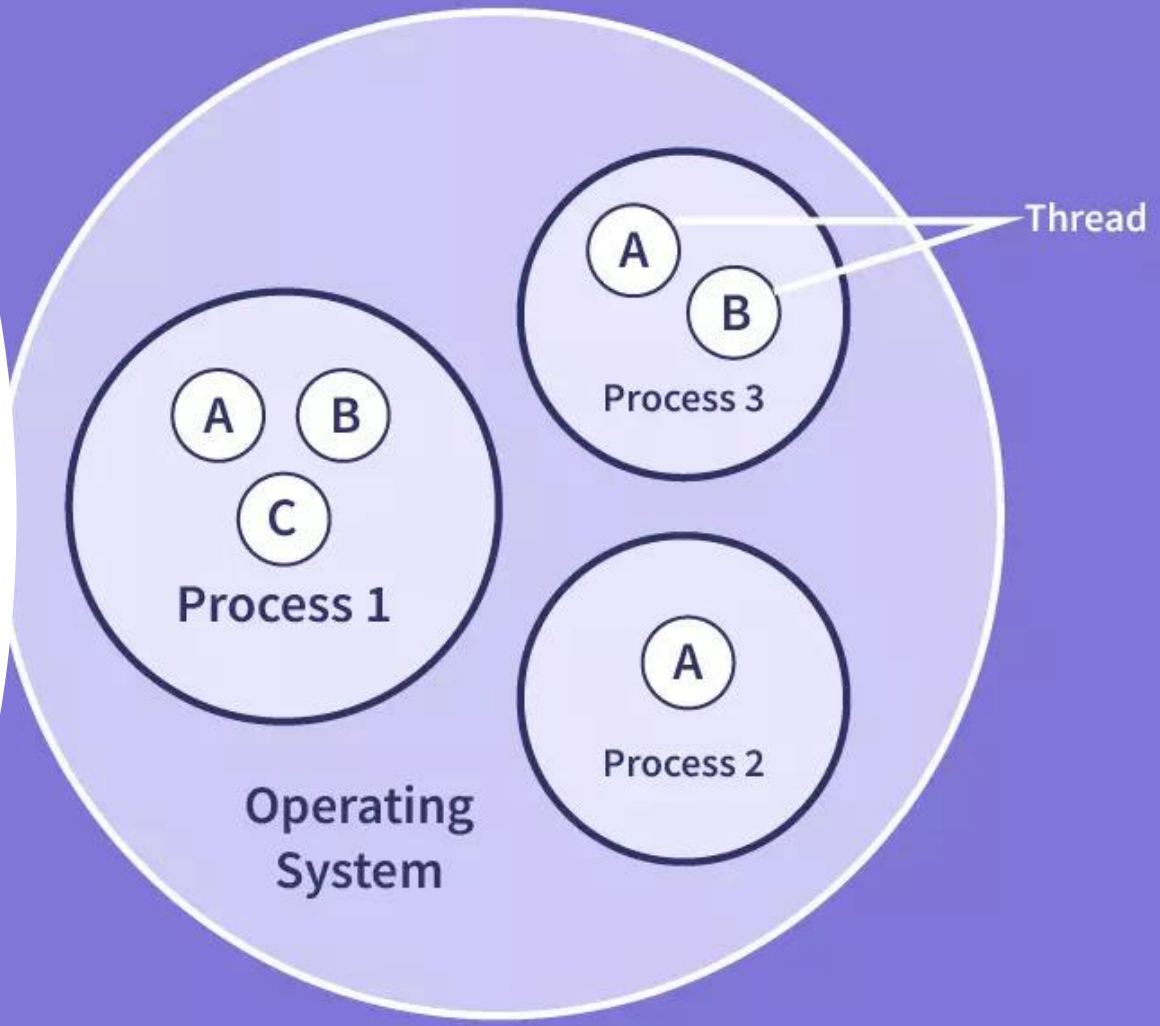
- In Java a Process is an independent execution environment created by operating System to run a particular java program.

```
Public class Process
{
   Public static void main(String[] args)
   {
      System.out.println("Hello World");
   }
}
```

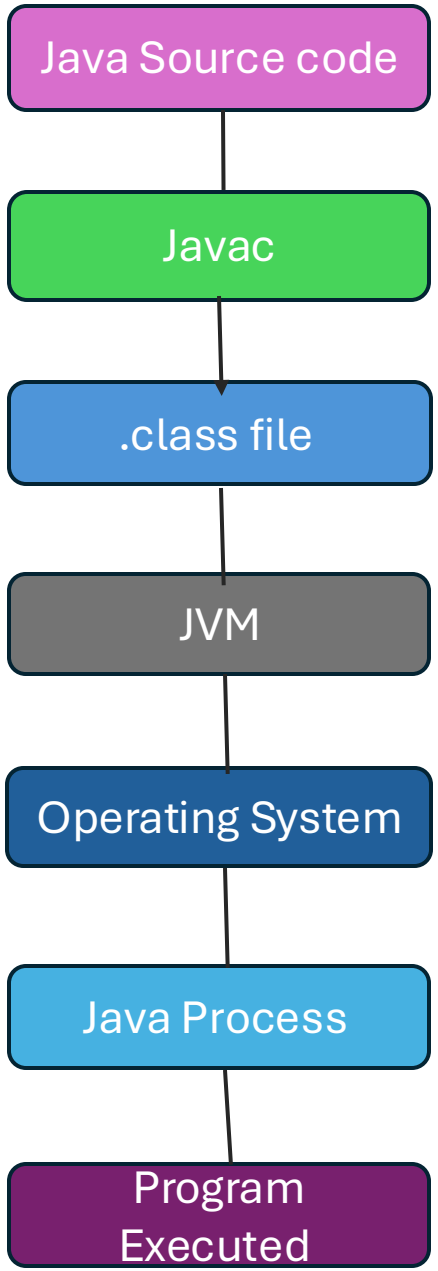
- Operating System creates a Process to execute this program.

Diagrammatically Representation of Process:-

Thread is an Part of Process . And Process is and part of Operating System (OS). It makes process for the sequentially execute the programs .

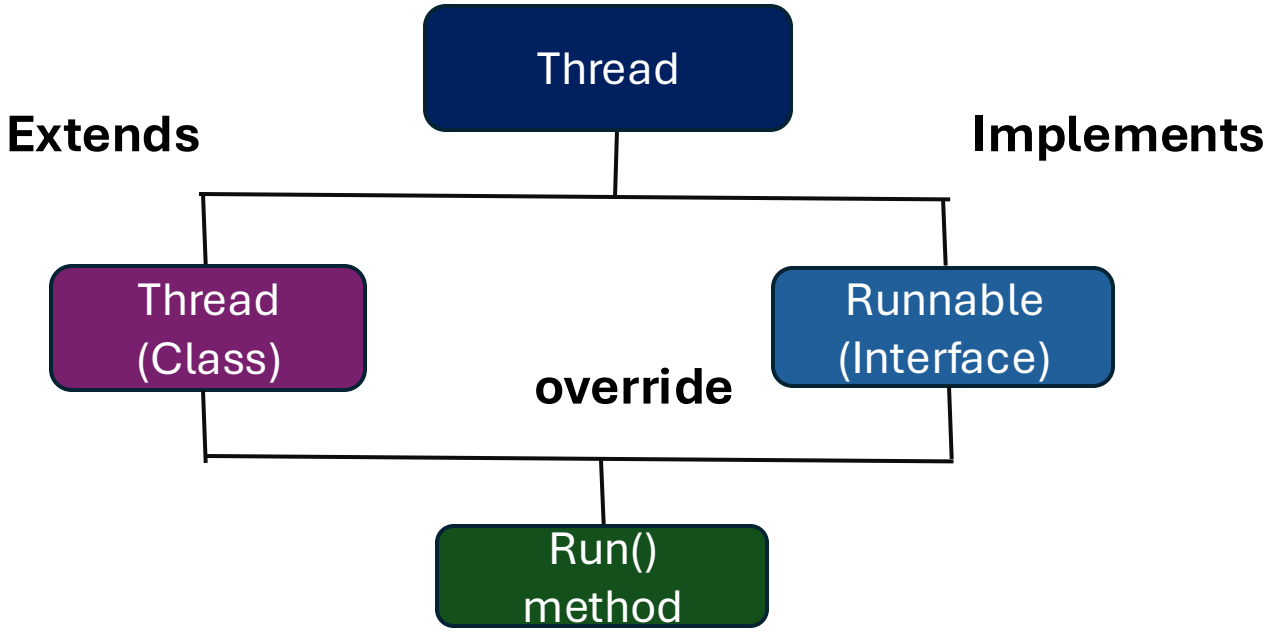


Process Workflow :-



Thread:-

- ✓ A thread is a smallest unit of execution within the process which allows a java program to perform multiple task concurrently.
- ✓ Threads are commonly used for tasks such as file operations, network request, and handling multiple clients requests.
- ✓ Java provides thread class and runnable interface for creating threads.



Thread Creation in java :-

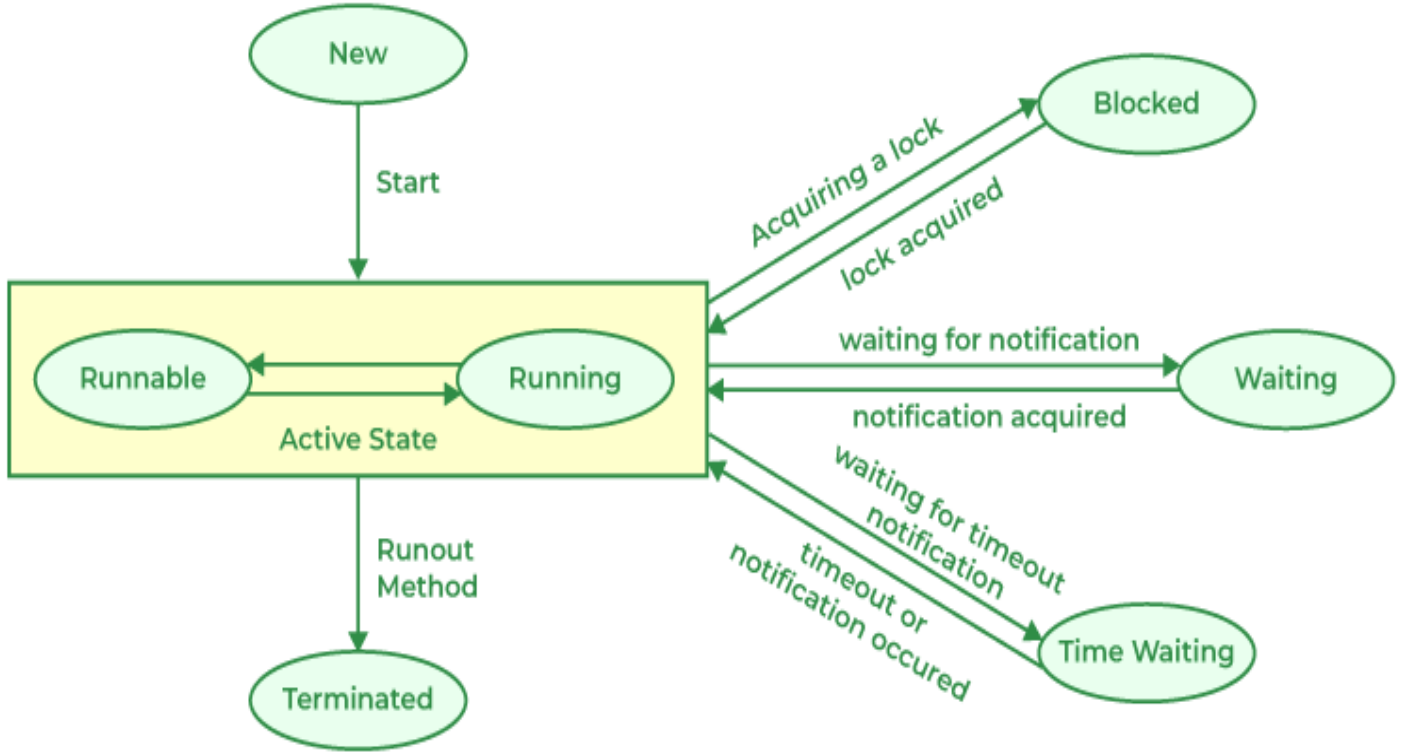
Creating a thread by extending thread class :

```
class Mythread extends Thread
{
   public void run()
   {
      System.out.println("Thread is running");
   }
}
public class Main
{
   public static void main(String[] args)
   {
      Mythread t1 = new Mythread();
      t1.start();
   }
}
```

Creating a thread by Implementing Runnable Interface :

```
class Mytask implements Runnable
{
   public void run()
   {
      System.out.println("Thread is running");
   }
}
public class Main
{
   public static void main(String[] args)
   {
      Mytask task= new Mytask();
      Thread t1 = new Thread(task);
      t1.start();
   }
}
```

Thread Lifecycle:-



States in Thread Lifecylce -

New State : A thread is in the NEW State when its thread object has been created but the start method has not yet been called.

Runnable State : When we call object name.start() , the thread moves from NEW to RUNNABLE. In java Runnable means the thread is ready to run or actually running.

Blocked State : A thread Enters the Blocked state when it is waiting to acquire a monitor lock , usually because another thread currently owns that lock .

For Example – Thread –01 is already inside the synchronized method if Thread –02 tries to enter in the same locked section then thread-02 become blocked.

Waiting State : A thread enters in waiting state when it waits indefinitely for another thread to perform some action .

Timed-Waiting State : A thread enters in Time-waiting when it waits for specific amount of time , i.e, Thread.sleep(2000).

Terminated State : When the thread completes its run() method or thread completes its task then its enters to the Terminated State .

Process vs Thread :

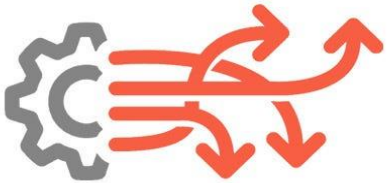
Process	Thread
Program in Execution	Part of process
Takes more time to create & Terminate	Takes Less time to create & Terminate
Does not share Data .	Share data with other threads .
Blocking one process doesn't affect others.	Blocking User-level thread may block all
More independent .	Depend on its own process.
A process contains multiple thread.	A thread belongs to a process.
Example : Running JVM / Application .	Example - main thread , worker thread

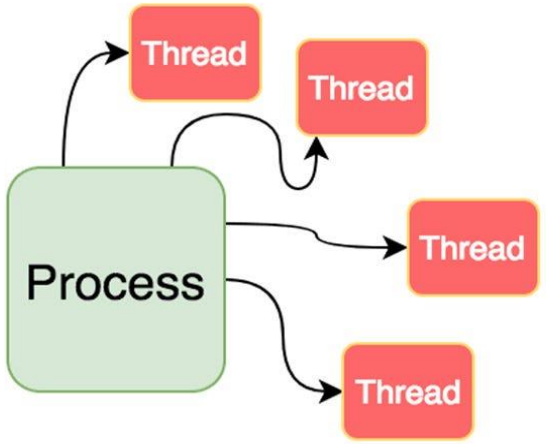
Topic :-02
"Multi-threading , synchronization , Inter-thread Communication"

Multi-threading :-

- Multi-threading in java is feature that enables program to run multiple threads simultaneously.
- It allowing tasks to execute in parallel and utilize the CPU more efficiently.
- A thread is lightweight, dependent unit of execution inside a program (process).
- It can maximize CPU Utilization.
- Thread share a common memory area(heap).







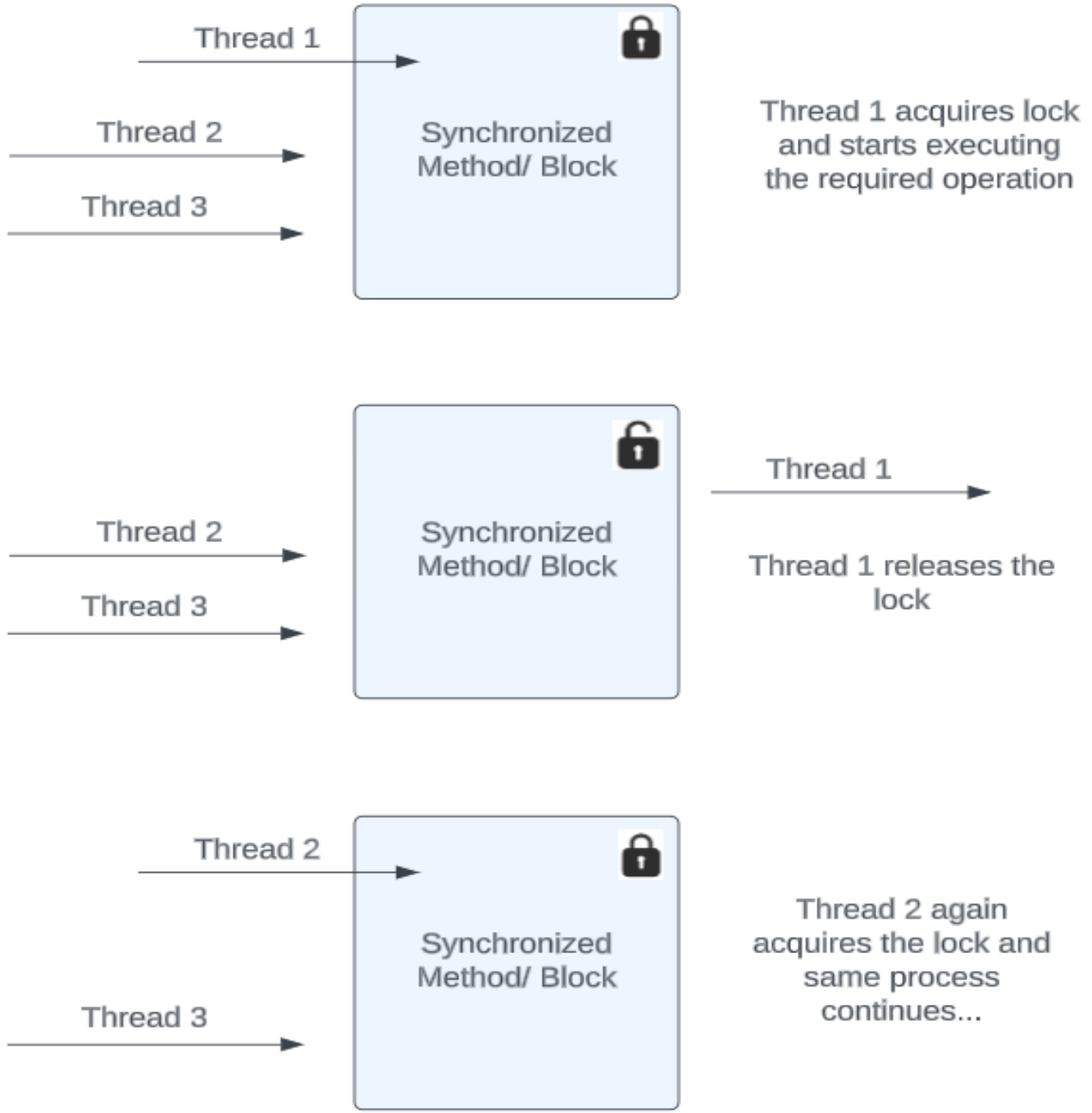
Practical implementation of Multi-threading in java :

```
class MyTask implements Runnable
{
   public void run()
   {
      System.out.println(Thread.currentThread().getName() +" is running");
   }
}
public class Main
{
   public static void main(String[] args)
   {
      MyTask task = new MyTask();
      Thread t1 = new Thread(task, "Thread-1");
      Thread t2 = new Thread(task, "Thread-2");
      Thread t3 = new Thread(task, "Thread-3");
```

```
        t1.start();
        t2.start();
        t3.start();
    }
}
```

Synchronization :

- ✓ Synchronization is a mechanism in java used to control access of multiple threads to shared resources.
- ✓ When multiple threads try to access or modify the same data at the same time, it can produce incorrect or inconsistent results. Synchronization helps prevents this problem .
- ✓ Synchronization ensures that's only one thread at a time can execute a synchronized section of code protected by the same lock.



Inter-Thread Communication :

Inter-thread communication is a mechanism that allows multiple threads to communicate and coordinate with each other when they are working on shared resources.

It is especially useful when one thread produce data and another thread consume data .

Ensures proper execution order among dependent threads.

It is mainly used when multiple threads depend on shared resources or need to work in a specific sequence.

Main methods used in inter-thread communication :

Wait() : Wait() causes the current thread to wait until another thread performs an appropriate notification, the thread remains waiting until it is notified and can reacquire the monitor.

Notify() : Wakes up one waiting thread (does not release lock immediately). In simpler words "something has changed. You can try to continue now".

Notify all() : Wakes up all waiting thread on the same object. In simpler words "something has changed. Everyone try to continue".

Topic:-03
"File Handling, File class, Byte streams , character Streams"

File handling :-

File handling in java means creating , reading , writing, updating and deleting files using java programs.

A java program can :

- Create a .txt file.
- Write a data into it .
- Read data from it.
- Add more data to it.
- Delete the file.
- Check whether a file exists.

Java provides file handling classes mainly through the java.io and java.nio.file packages.

Basic file handling operations :

Create : Create a new file.

Write : put data into it.

Read : Get data from it.

Update : Add/change existing data.

Delete: Remove a file.

File Class :-

Used to represent a file or directory and perform operations such as:

- Check whether file exists
- Create file
- Delete file
- Get file name
- Get file path
- Check file size

Package :

Import java.io.File;

FileWriter() : Used to write character/text data into a file.

Required Library to be import - import java.io.FileWriter;

FileReader() : Used to read character/text data from a file.

Required Library to be import - import java.io.FileReader;

BufferedReader() : Used to read text efficiently, usually line by line.

Required Library to be import - import java.io.BufferedReader;

Bufferedwriter() : Used to write text efficiently.

Required Library to be import - import java.io.BufferedWriter;

Creating a File :

```
import java.io.File;
import java.io.IOException;

public class Main {
  public static void main(String[] args) {

    File file = new File("student.txt");
    try {
      if (file.createNewFile()) {
        System.out.println("File created successfully.");
      } else {
        System.out.println("File already exists.");
      }
    } catch (IOException e) {
      System.out.println("An error occurred.");
    }}}
```

Writing a file :

```
import java.io.FileWriter;
import java.io.IOException;


public class Main {
   public static void main(String[] args) {


      try {
         FileWriter writer = new FileWriter("student.txt");


         writer.write("Name: Dhruvil\n");
         writer.write("Course: Java\n");
         writer.write("Subject: File Handling");


         writer.close();


         System.out.println("Data written successfully.");
```

```
 }
        catch (IOException e)
        {
          System.out.println("An error occurred.");
       }
   }
}
```

Reading from file :

```
import java.io.FileReader;
import java.io.IOException;


public class Main {
   public static void main(String[] args) {
    try {
          FileReader reader = new FileReader("student.txt");
```

```
        int data;
        while ((data = reader.read()) != -1) {
            System.out.print((char) data);
        }
        reader.close();
      } catch (IOException e) {
        System.out.println("An error occurred.");
      }
    }
}
```

Reading File Line-by-line (BufferedReader) :

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class Main {
```

```
    public static void main(String[] args)
    {
     Try
     {
        BufferedReader reader = new BufferedReader(new FileReader("student.txt"));
        String line;
        while ((line = reader.readLine()) != null) {
           System.out.println(line);
       }
            reader.close();
       }
         catch (IOException e)
         {
            System.out.println("An error occurred.");
         }
}
}
```

Byte Streams : -

- Byte is the smallest unit of data consisting of 8-bit : i.e, 1 Byte = 8-bit.
- A byte streams in java is a stream that handles input and output of raw 8-bit binary data.
- It is mainly used for reading and writing non-text data such as images, audio, video, or any binary file.

Main Byte Stream classes :

There are mainly two important classes are -

- **FileInputStream :** Used to read bytes from a file .

  Import java.util.FileInputStream;
- **FileOutputStream :** Used to write bytes from a file.

  Import java.util.FileOutputStream;

Practical implementation of Byte Streams in java :

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class Main
{
      public static void main(String[] args)
      {
     try {
        // Write data into file
        FileOutputStream output =
              new FileOutputStream("data.txt");

        String message = "Hello Java";

        output.write(message.getBytes());

        output.close();
```

```
// Read data from file
        FileInputStream input =
                new FileInputStream("data.txt");


        int data;


        while ((data = input.read()) != -1) {
           System.out.print((char) data);
        }


        input.close();


    } catch (IOException e) {
        System.out.println("File error occurred.");
    }
  }
```

Character Streams :-

A character Streams handle data character - by - character .

Character Streams are used to read and write character / text data in java.

They are mainly used for file containing text , such as : " .txt " , " .csv " , " .java " , " .html " etc .

Main character Streams Classes:

There are mainly two important classes are -

FileReader : Used to read text from file .

Filewriter : Used to Write data into file.

Practical implementation of Character streams in java :

```
import java.io.FileWriter;
import java.io.FileReader;
import java.io.IOException;


public class Main
{
   public static void main(String[] args)
   {
      try {
         // Write data
         FileWriter writer =
               new FileWriter("data.txt");
              writer.write("Hello Java");
              System.out.println("Data written successfully");
              writer.close();
```

```
        // Read data
        FileReader reader = new FileReader("data.txt");
        int data;
        while ((data = reader.read()) != -1) {
            System.out.print((char) data);
        }
        reader.close();
      } catch (IOException e)
        {
        System.out.println("File error occurred.");
      }
   }
}
```

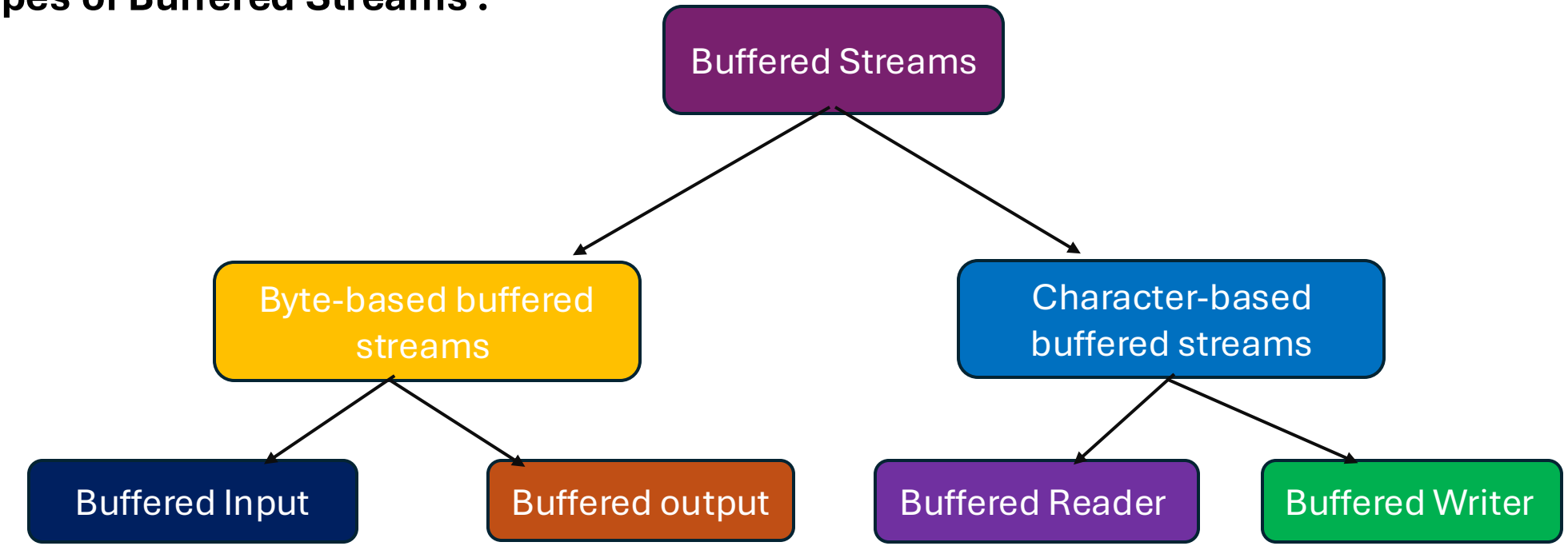
Topic : -04
"Buffered Streams, serialization, deserialization "

Buffered Streams : -

Buffered Streams are used to make reading and writing data faster and more efficient by using a temporary memory area called a buffer.

A buffer is temporary memroy area that stores data while reading from or writing to a file.

Types of Buffered Streams :



❑**Byte-based buffered streams : -**

It is mainly used for binary data.

BufferedInputStream.

BufferedOutputStream.

Examples :- images , video , zip , pdf, audio .

BufferedInputStreams – BufferedInputStreams is used to efficiently read binary files .

Practical implementation :

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

public class Main {
public static void main(String[] args)
{
    try {
```

```
      BufferedInputStream input  =  new BufferedInputStream( new FileInputStream("photo.jpg"));
      int data;
      while ((data = input.read()) != -1) {
          // Process byte
       }
       input.close();
       System.out.println("File read successfully.");

       }
       catch (IOException e) {
       System.out.println("File error occurred.");
     }
   }
}
```

BufferedOutputStreams – BufferedOutputStreams is used to efficiently write binary data .

Practical implementation:

```
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class Main {
   public static void main(String[] args)
   {
      try {
      BufferedOutputStream output = new BufferedOutputStream(new FileOutputStream("data.txt"));
         String message = "Hello Java";
         output.write(message.getBytes());
         output.close();
         System.out.println("Data written.");

      } catch (IOException e) {

         System.out.println("File error occurred.");
      }
   }
}
```

Character-based Streams:-

It is mainly used read text data.

BufferedReader.

BufferedWriter.

Example :- TXT, CSV, HTML.

BufferedReader :- BufferedReader is used to read text efficiently , Especially line-by-line

Practical Implementation :

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
public class Main {
   public static void main(String[] args) {
      try {
         BufferedReader reader (new BufferedReader(new FileReader("data.txt"));
         String line;
        while ((line = reader.readLine()) != null) {

            System.out.println(line);
         }
         reader.close();
         } catch (IOException e) {
```

```
System.out.println("File error occurred.");
      }
   }
}
```

BufferedWriter : - BufferedWriter is used to Write text efficiently.

Practical Implementation:

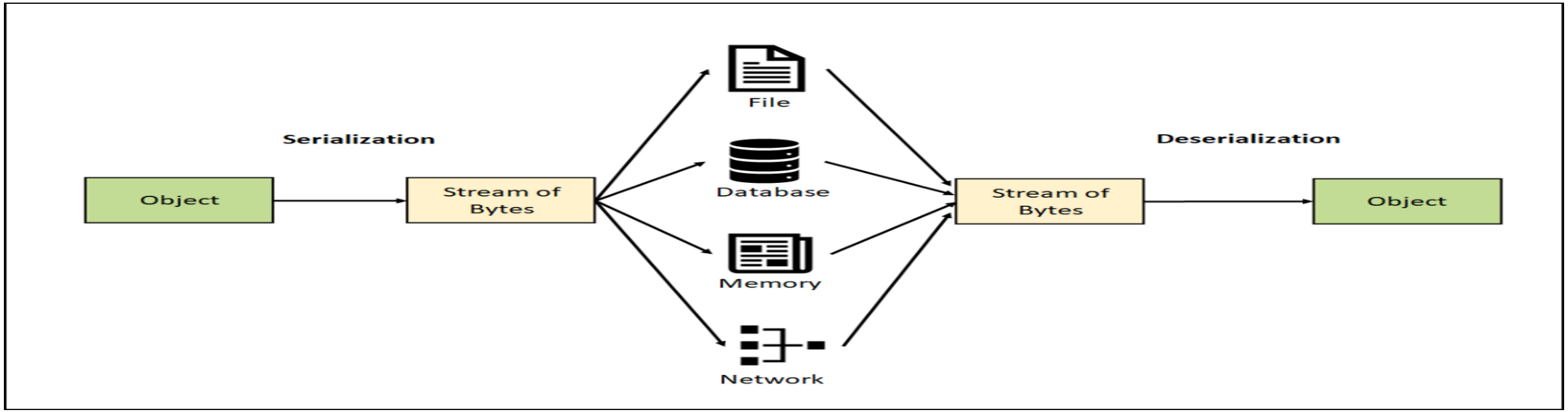
```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
public class Main {
   public static void main(String[] args) {
      try {
         BufferedWriter writer = new BufferedWriter(new FileWriter("data.txt"));
         writer.write("Hello Java");
         writer.newLine();
          writer.write("Welcome Students");
         writer.close();
```

```
System.out.println("Data written.");
        } catch (IOException e) {
            System.out.println("File error occurred.");
        }
    }
}
```

Serialization :-

- ✓ Serialization is the process of converting an object's state into a byte streams.
- ✓ Allowing it to be saved to a file, stored in database or transmitted over the network.
- ✓ The generated byte streams is platform independent , an object serialized on one platform can be decentralized on another platform.
- ✓ Implemented using serializable marker interface and importing ObejctOutputStreams Library.
- ✓ Facilitated object transfer across networks in distributed applications.

Diagrammatically Representation of Serialization :



Deserialization : -

- ✓ Deserialization is the reverse process of Serialization , It convert a byte streams back into its original java object.
- ✓ This allows previously serialized objects to be retrieved and used within the application.
- ✓ It will implemented by importing ObjectInputStream library .

Topic:-05
"Lambda Expression, Stream API, Optional class"

Lambda Expression :-

- Lambda expression is introduced in Java 8 mainly used to make code shorter , cleaner , and easier to use with functional interface and collections.
- It is short block of code that acts as a anonymous function – method that does not need a name, a return type, or a class wrapper.

Syntax of declare lambda expression :

(parameter) -> { body }.

It consist of three parts :

Parameter List – The input argument required by method.

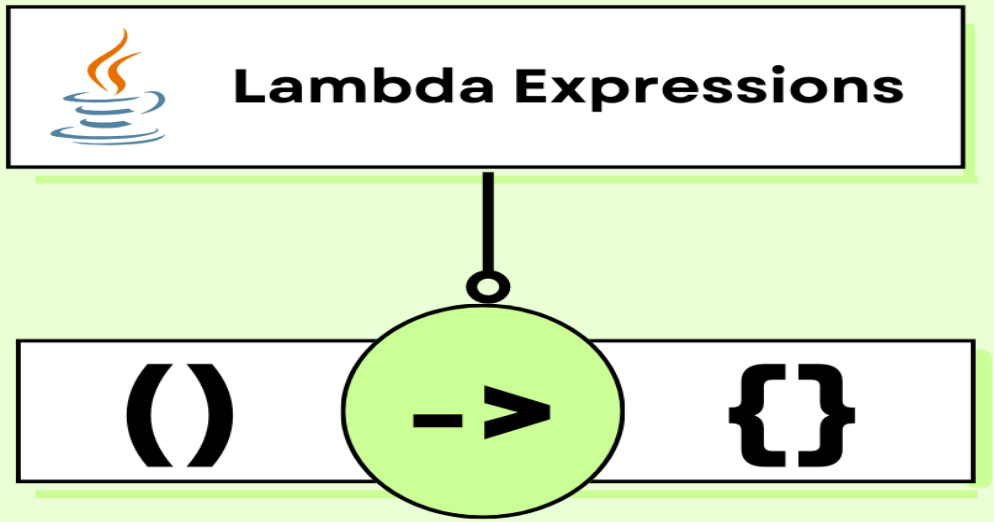
Arrow Token(->) - Seperates the parameters from the execution logic.

Body – Contains the expression or statements that execute.

Example : -

(a, b) -> {

```
Int result = a*b;
Return result ;
}
```



Stream API:-

Stream API is introduced in java 8 , it is powerful tool designed to process collections of data in a declarative and functional programming style.

Instead of writing Complex loops (for or while) and managing temporary variables, we can chain methods together to perform operations such as

Filtering.

Transforming.

Aggregate data.

It is useful for operations such as :

- Filtering data
- Sorting data
- Searching data
- Transforming data
- Calculating results
- Collecting results
- Processing data with Lambda Expressions

Optional class :-

Optional class is also introduced in java 8.

It is created to handle null values safely . It is a container object that may or may not contain a value .

It helps to avoid NullPointerException and improve code readability.

Provides methods to check and process values easily.

It uses following methods :

Optional.of() : It is used when we sure that value is not null.

Optional.ofNullable() : It is used when the value may be null.

Optional.empty() : Creates an empty Optional.

Ispresent() : Checks whether a value exist .

Get() : Used to retrieve the value inside optional.

----------Syllabus Completed----------